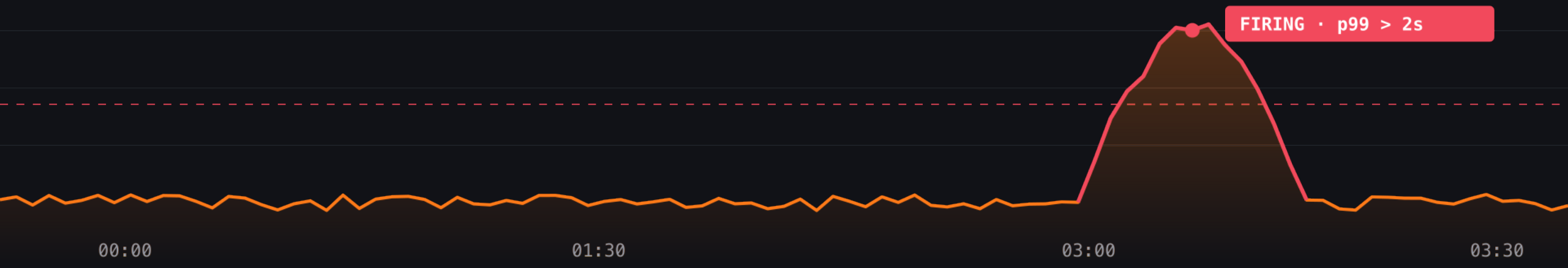
CLOUD NATIVE MUMBAI × GRAFANA & FRIENDS MUMBAI

The 3 AM Page.

A hands-on observability workshop with Docker, Prometheus & Grafana.

Afzal Hassan · 17 Oct 2026 · Mumbai



$ whoami

Afzal Hassan

Platform & Reliability Engineer · Sunfire Technologies

01 **Runs air-gapped OpenShift for a Tier-1 bank**
zero internet egress, DR drills, zero-trust policies

02 **Co-organizer, Grafana & Friends Mumbai**
core team, Cloud Native Mumbai

03 **CKA · builds OutagePilot**
an AI SRE that turns K8s anomalies into runbooks

Scan → connect
in/iemafzalhassan

BEFORE WE START

Is your laptop ready?

```
$ git clone https://github.com/iemafzalhassan/the-3am-page
$ cd the-3am-page && ./preflight.sh
  ✔️ Docker 29.7.2
  ✔️ Compose 5.5.1
  ✔️ 7.6 GB memory for Docker
  ✔️ ports 8080 9090 3000 5001 free
  ✔️ images ready (619 MB)
✅ Ready for 17 Oct.
```

No internet? Grab a pen drive: `./offline/load-bundle.sh`



Green on the lid

done, all good — I can move on



Red on the lid

stuck — a volunteer comes to you

The repo

github.com/iemafzalhassan/the-3am-page

Source: Docker Compose v5.5.1 release ↗

QUICK POLL · HANDS UP

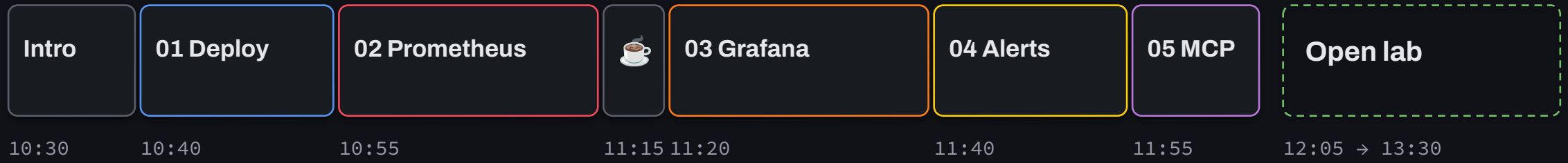
01 Hands up if you've …run docker compose up

02 Hands up if you've …written a PromQL query

03 Hands up if you've …been woken up by an alert

TODAY

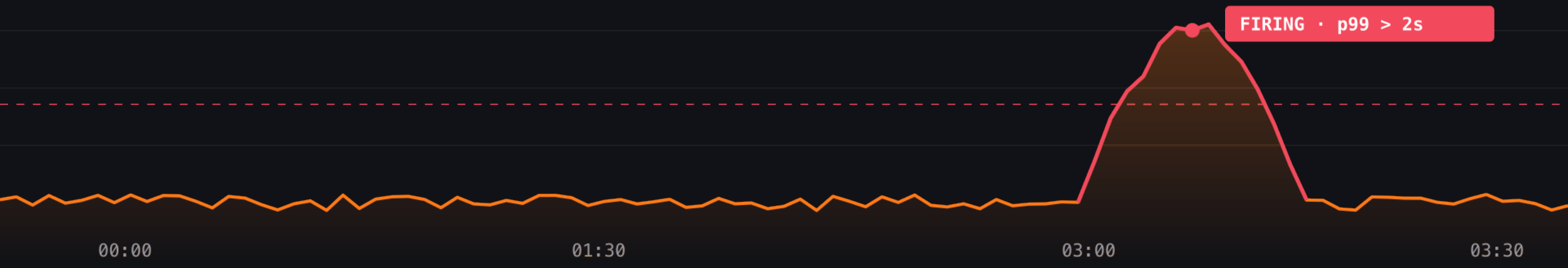
Four modules. Four checkpoints.



Fell behind? `./checkpoint.sh N` and you're back with everyone.

03:07 AM

Checkout is slow.
Customers are angry.
All you have is docker logs.



TWO WORDS PEOPLE MIX UP

Monitoring vs observability

MONITORING

Is it broken?

- → Known unknowns
- → Checks you wrote in advance
- → Green / red

OBSERVABILITY

Why is it broken?

- → Unknown unknowns
- → New questions, no redeploy
- → Shape, not just status

Monitoring is something you do. Observability is something your system has.

Source: OpenTelemetry: Observability primer ↗ · Grafana Labs: What is observability? ↗

THREE SIGNALS

Metrics first. Here's why.

TODAY

Metrics

Numbers over time. Cheap, aggregatable, alertable. Tells you THAT something is wrong.

LOKI

Logs

Events with detail. Tells you WHAT happened to one request.

TEMPO · OTEL

Traces

One request's journey across services. Tells you WHERE time went.

Source: OpenTelemetry: What is OpenTelemetry? ↗ · Prometheus docs: OTLP receiver ↗

01

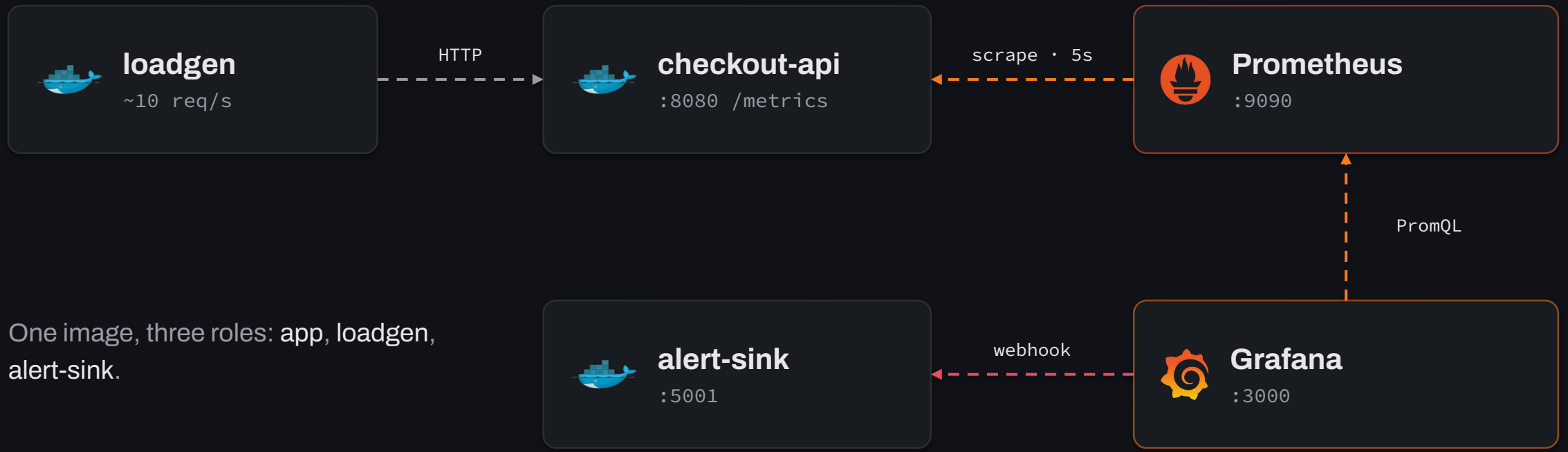
Deploy

Make it run. Then make it talk.

10:40 · 15 MIN

WHAT WE'RE RUNNING

Five containers. One story.



One image, three roles: app, loadgen, alert-sink.

Source: Prometheus docs: Overview (pull over HTTP) ↗ · Grafana docs: Alerting fundamentals ↗

FOLLOW ALONG

STEP 1.1

Bring it up

```
$ docker compose up -d
✔ Container the-3am-page-checkout-api-1  Started
✔ Container the-3am-page-loadgen-1       Started
✔ Container the-3am-page-prometheus-1    Started
✔ Container the-3am-page-grafana-1       Started
✔ Container the-3am-page-alert-sink-1    Started
$ docker compose ps --format "table {{.Service}}\t{{.Status}}"
```

OPEN IN YOUR BROWSER

localhost:8080/products
the app

localhost:9090
Prometheus

localhost:3000
Grafana · admin / admin

localhost:5001
alert-sink

Source: Docker docs: Networking in Compose ↗

CHECKPOINT 1 · ./checkpoint.sh 1

COMPOSE.YAML

The whole stack fits on one slide

compose.yaml (excerpt)

```yaml
services:
  checkout-api:
    image: ghcr.io/iemafzalhassan/checkout-api:1.0
    build: ./app
    ports: ["127.0.0.1:8080:8080"]
  loadgen:
    image: ghcr.io/iemafzalhassan/checkout-api:1.0
    command: ["python", "loadgen.py"]
  prometheus:
    image: prom/prometheus:v3.15.0
    volumes: ["./prometheus:/etc/prometheus:ro"]
  grafana:
    image: grafana/grafana:13.2.4
    volumes: ["./grafana/provisioning:/etc/grafana/provisioning:ro"]
```

01 Same image, new command

loadgen and alert-sink reuse the app image.

02 Config is mounted

Edit a file, restart. Nothing baked in.

03 Pinned versions

Never :latest in a workshop — or in prod.

04 127.0.0.1 only

Shared wifi: nobody can open your Grafana.

Source: Docker docs: Compose ports (host IP binding) ↗ · Docker docs: Pin image versions ↗

FOLLOW ALONG

STEP 1.2

This is all Prometheus ever sees.

```
$ curl -s localhost:8080/metrics | grep http_requests_total
# HELP http_requests_total Total HTTP requests served.
# TYPE http_requests_total counter
http_requests_total{method="GET",route="/products",status="200"} 18231.0
http_requests_total{method="POST",route="/checkout",status="500"} 42.0
```

name

what is measured

labels

which slice of it

value

the number right now

Source: Prometheus docs: Exposition formats ↗ · Prometheus docs: up metric ↗

02

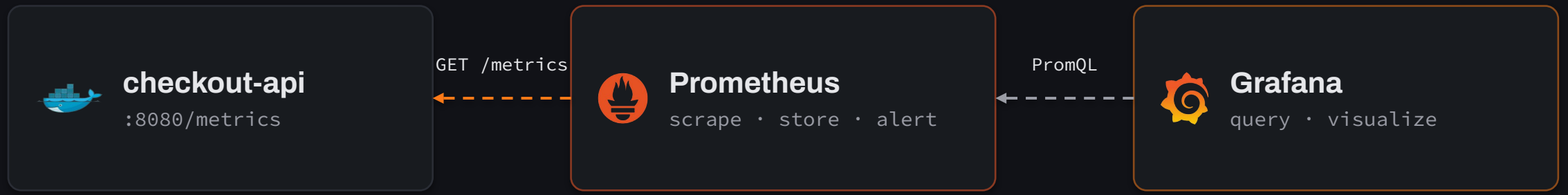
Prometheus

Collect. Store. Ask questions.

10:55 · 20 MIN

THE PULL MODEL

Prometheus pulls.
Your app just has to be ready.

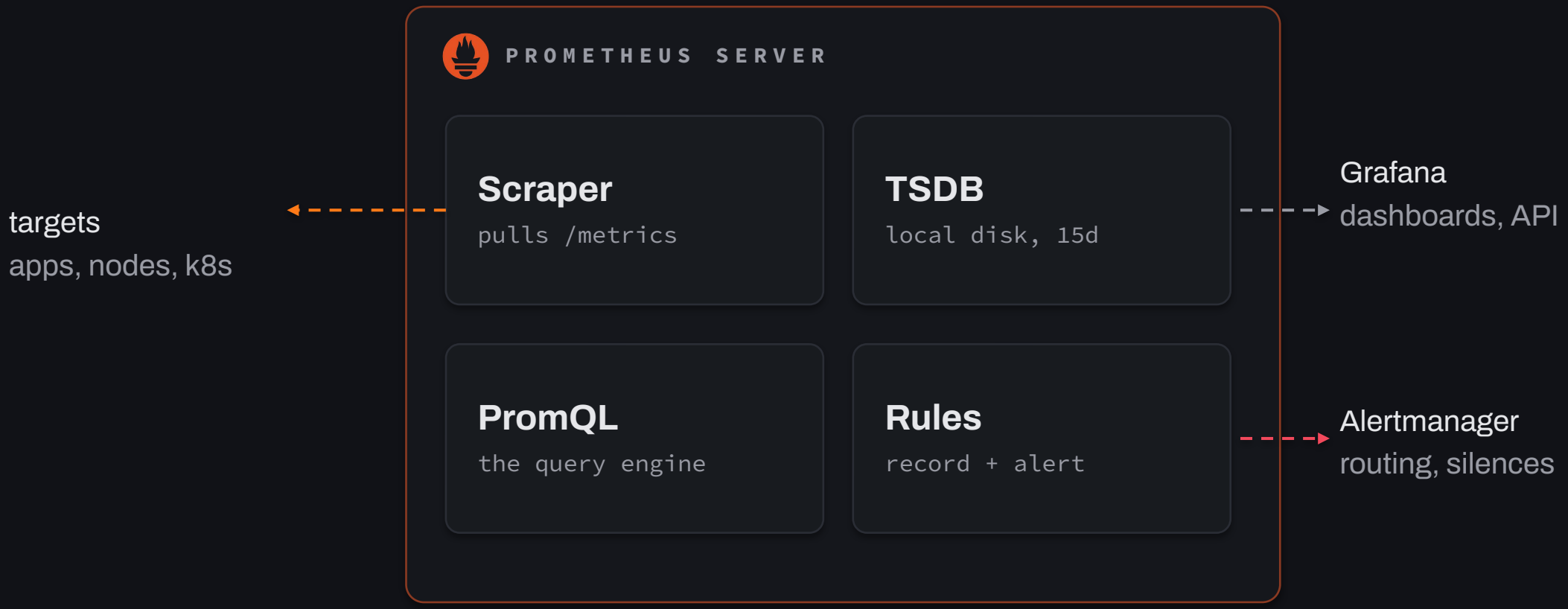



```
# Prometheus asks every 5s.  The app never knows Prometheus exists.
up{job="checkout-api", instance="checkout-api:8080"}  1
```

Source: Prometheus docs: Overview (pull model) ↗ · Prometheus docs: When to use the Pushgateway ↗

INSIDE THE BOX

One binary, four jobs



Source: Prometheus docs: Storage ↗ · Prometheus docs: Alerting rules ↗

PROMETHEUS/PROMETHEUS.YML

Eleven lines of config

prometheus.yml

```yaml
global:
  scrape_interval: 5s
  evaluation_interval: 5s

rule_files:
  - rules.yml

scrape_configs:
  - job_name: checkout-api
    static_configs:
      - targets: ["checkout-api:8080"]
```

01 **5s is a workshop number**

15–60s is normal in production.

02 **Docker DNS**

checkout-api resolves to the container.

03 **Kubernetes?**

kubernetes_sd_configs or a ServiceMonitor.

Source: Prometheus docs: Configuration (defaults) ↗ · Prometheus docs: kubernetes_sd_config ↗

FOLLOW ALONG

STEP 2.1

Is anyone home?

```
→ browser: http://localhost:9090/targets
checkout-api (1/1 up)   http://checkout-api:8080/metrics   UP   last scrape 2.1s ago
```

```
up
up{instance="checkout-api:8080", job="checkout-api"}   1
```

`up` is free. Prometheus writes it on every scrape. Your first alert should be up == 0.

Source: Prometheus docs: up metric ↗

THE DATA MODEL

Name + labels = one time series

```
http_requests_total{method="POST", route="/checkout", status="500"}  42  @1792186620
```

metric name — **labels — the dimensions** — **value** — **timestamp**

Change any label value and you get a different series. That's the superpower and the trap.

Source: Prometheus docs: Data model ↗

FOUR METRIC TYPES

Pick the right shape

COUNTER

Only goes up

requests, errors, orders.
Always wrap in rate().

GAUGE

Up and down

in-flight requests, memory, queue depth. Use as-is.

HISTOGRAM

Buckets

latency, sizes. Quantiles at query time — aggregates across pods.

SUMMARY

Client quantiles

Can't aggregate across replicas. Prefer histograms.

Source: Prometheus docs: Metric types ↗ · Prometheus docs: Histograms and summaries ↗

INSTRUMENTATION

Five lines. One metric. Zero magic.

simplified from app/main.py

```python
from prometheus_client import Histogram

LATENCY = Histogram(
    "http_request_duration_seconds",
    "Time spent serving a request",
    ["method", "route"],
)

@LATENCY.labels("POST", "/checkout").time()
def checkout(cart):
    ...
```

✗

grep-ing logs at 3 AM, hoping

✓

```
histogram_quantile(0.99, …)
```

Source: prometheus_client (Python): Histogram ↗

CARDINALITY

Every label value is a new time series.

✓ `3 methods × 10 routes × 5 statuses`

up to 150 series

✗ `… × user_id (1,000,000 users)`

up to 150,000,000 series



meme: Gru's Plan · imgflip

Source: Prometheus docs: Naming, labels ↗ · Prometheus docs: Do not overuse labels ↗

Never graph a raw counter

```
http_requests_total{route="/checkout"}
```



Only goes up. The dip is a restart.

```
rate(http_requests_total{route="/checkout"}[1m])
```



Requests per second, right now.

Source: Prometheus docs: rate() ↗ · Prometheus docs: irate() ↗

PROMQL · 2

What % of requests fail?

```
  sum by (route) (rate(http_requests_total{status=~"5.."}[5m]))
/
  sum by (route) (rate(http_requests_total[5m]))
```

01 **=~ "5.."**
regex: every 5xx status

01 **sum by (route)**
keep route, add up the rest

01 **→ 0.01**
1% on /checkout: the app's baseline

Source: Prometheus docs: Regular expressions ↗ · Prometheus docs: Aggregation operators ↗

How slow is slow? Ask p99.

```
histogram_quantile(0.99,
  sum by (le, route) (
    rate(http_request_duration_seconds_bucket[5m])
  )
)
```

01 **0.99**

1 in 100 requests is slower than this.

02 **keep le**

Drop it and the maths silently breaks.

03 **_bucket**

Histograms expose one counter per bucket.

Averages hide your angriest customers. Percentiles don't.

Source: Prometheus docs: histogram_quantile() ↗ · Prometheus docs: Errors of quantile estimation ↗

YOUR TURN • 4 MINUTES

STEP 2.2

Three questions. Answer them in PromQL.

Q1

How busy?

Requests per second hitting /products right now.

Q2

How broken?

What fraction of /checkout calls fail?

Q3

What sells?

Most used payment method in the last 5 minutes.

Answers go up on screen after 4 minutes. Ask your neighbour first.

Source: Prometheus docs: increase() ↗ · Prometheus docs: Aggregation operators (topk) ↗

CHECKPOINT 2 • ./checkpoint.sh 2

11:15 · CATCH-UP

Five minutes. Catch up.

```
$ docker compose up -d
$ ./checkpoint.sh 2
# Windows: .\checkpoint.ps1 2
```

Red sticky? Keep it up — a volunteer is coming.



meme: Left Exit 12 Off Ramp · imgflip

03

Grafana

See it. Share it. Version it.

11:20 · 20 MIN

GRAFANA IN 30 SECONDS

Grafana stores no metrics. It asks.

DATA SOURCES

Ask anything

Prometheus, Loki, Tempo, SQL, cloud APIs — 100+ plugins.

PANELS

Show it

Time series, stat, gauge, heatmap, table.

VARIABLES

Reuse it

One dashboard for every service and route.

ALERTING

Act on it

Rules, contact points, notification policies.

Already wired for you: `grafana/provisioning/datasources/prometheus.yml`

Source: Grafana docs: Data sources ↗ · Grafana plugin catalog: data sources ↗

THE RED METHOD

Three panels answer "is it us?"

R

Rate

```
sum(rate(http_requests_total[1m]))
```

E

Errors

```
sum(rate(http_requests_total{status
=~"5.."}[1m]))
```

D

Duration

```
histogram_quantile(0.99, …)
```

Resources instead of services? Use USE: Utilization, Saturation, Errors.

Source: Tom Wilkie: The RED Method ↗ · Brendan Gregg: The USE Method ↗

FOLLOW ALONG

STEP 3.1

Build it: three panels

1 **New dashboard → Add visualization → Prometheus**

2 **Requests/s**

```
sum by (route) (rate(http_requests_total[1m]))
```

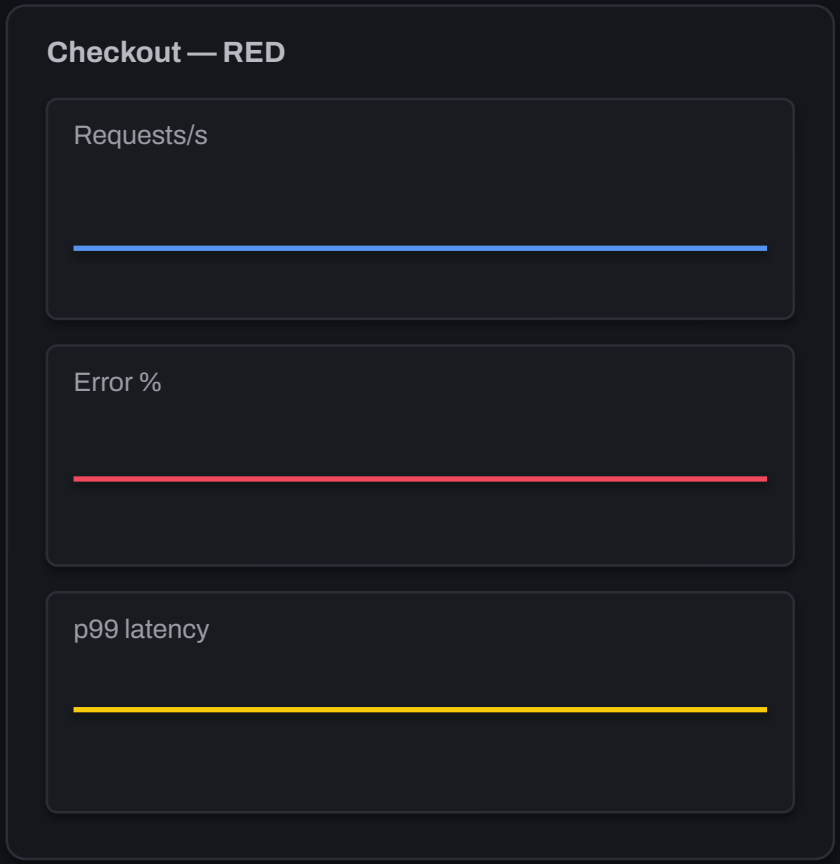
3 **Error %**

```
sum(rate(http_requests_total{status=~"5.."}[1m])) /
sum(rate(http_requests_total[1m]))
```

4 **p99 latency**

```
histogram_quantile(0.99, sum by (le, route)
(rate(http_request_duration_seconds_bucket[1m])))
```

5 **Save as "Checkout — RED"**



Source: Grafana docs: Prometheus query editor (legend) ↗ · Grafana docs: Standard options (units) ↗

VARIABLES

One dashboard, every route

Variable: route · type: Query · multi-value + include All

Query type: Label values · Label: route · Metric: http_requests_total

Panel query

```
sum by (route) (rate(http_requests_total{route=~"$route"}[1m]))
```

Multi-value or "All"? Use =~ — not =. Grafana sends a regex.

Source: Grafana docs: Prometheus template variables ↗ · Grafana docs: Multi-value variables ↗

DASHBOARDS AS CODE

If it isn't in git, it doesn't exist.

grafana/provisioning/dashboards/workshop.yml

```yaml
apiVersion: 1
providers:
  - name: workshop
    folder: The 3 AM Page
    type: file
    allowUiUpdates: true
    options:
      path: /var/lib/grafana/dashboards
```

01 **Export**

Export → Export as code → Download file.

02 **Commit**

grafana/dashboards/checkout-red.json

03 **Grow up**

Foundation SDK, Terraform, Grafonnet.

Source: Grafana docs: Provisioning dashboards ↗ · Grafana docs: Export a dashboard as code ↗

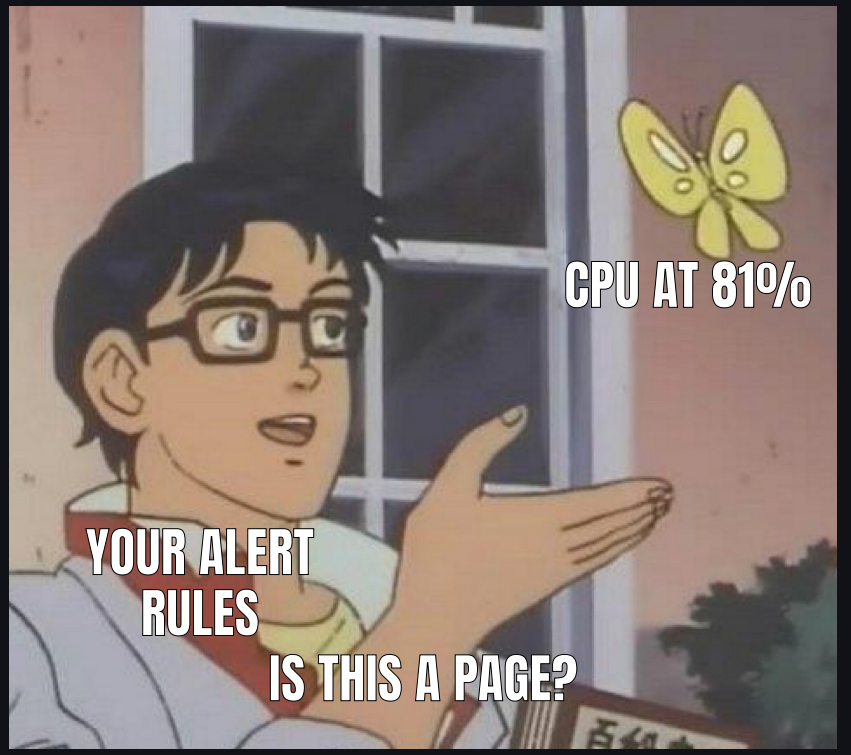
CHECKPOINT 3 · ./checkpoint.sh 3

04

Alerts

Wake the right person. For the right reason.

11:40 · 15 MIN



meme: Is This A Pigeon · imgflip

ALERT LIFECYCLE

Inactive → Pending → Firing



for: 1m means "true for one full minute, without a break". One bad scrape no longer wakes anyone up.

Source: Prometheus docs: Alerting rules (for) ↗ · Prometheus docs: Inspecting alerts ↗

PROMETHEUS / RULES.YML

An alert, line by line

rules.yml

```yaml
- alert: HighLatencyP99
  expr: |
    histogram_quantile(0.99, sum by (le) (
      rate(http_request_duration_seconds_bucket{route="/checkout"}[1m])
    )) > 1
  for: 1m
  labels:
    severity: page
  annotations:
    summary: "Checkout p99 is above 1s"
    runbook_url: "https://wiki/runbooks/checkout-latency"
```

01 **expr**
the question

02 **for**
the patience

03 **labels**
who gets woken up

04 **annotations**
what they read at 3 AM

Source: Prometheus docs: Alerting rules ↗ · Prometheus docs: Alertmanager ↗

GOOD ALERTS

Page on symptoms, not causes

✓ Page a human

→ Error rate above SLO

→ p99 latency above target

→ Service unreachable (up == 0)

✗ Dashboard or ticket

→ CPU at 80%

→ One pod restarted

→ Disk 70% full

Every page needs: user impact · a human action · a runbook link

Source: Google SRE book: Symptoms vs causes ↗ · Google SRE workbook: Multiwindow, multi-burn-rate alerts ↗

BREAK IT

STEP 4.1

Predict the graph.

```
$ curl -X POST localhost:8080/chaos -d '{"latency_ms": 1200}'
```

A

p99 jumps instantly. Alert fires instantly.

B

p99 jumps in seconds. Pending at once, Firing a minute later.

C

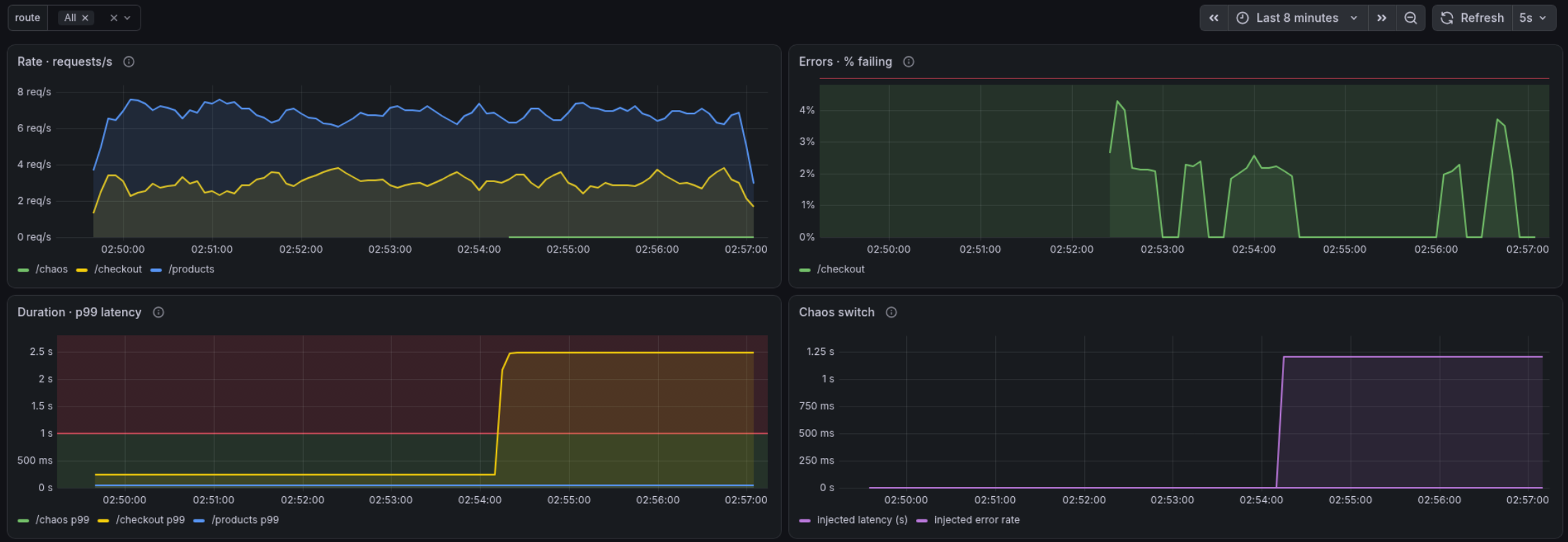
Nothing visible — the fast /products traffic hides it.

Source: Prometheus docs: Alerting rules (for) ↗ · Prometheus docs: rate() ↗

CHECKPOINT 4 · ./checkpoint.sh 4

WHAT ACTUALLY HAPPENED

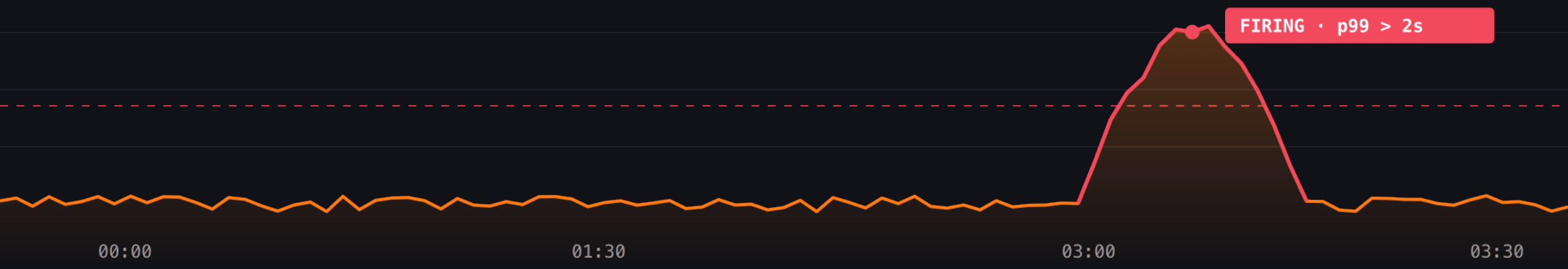
Measured, not predicted.



Test run, 16 Oct: chaos on at 02:54 → /checkout p99 ≈ 2.5 s → CheckoutP99High firing on alert-sink.

03:07 AM, AGAIN

This time, you saw it coming.



05

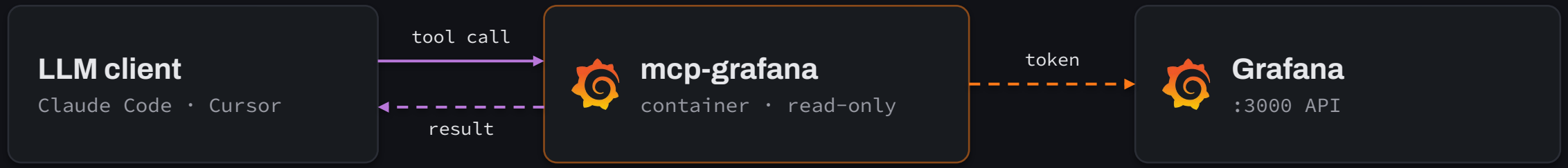
Bonus: MCP

Let the LLM read your dashboards.

11:55 · 10 MIN · SPEAKER DEMO

MODEL CONTEXT PROTOCOL

MCP in one picture



TOOLS THE MODEL CAN CALL

`search_dashboards` `get_dashboard_by_uid` `list_datasources`

`query_prometheus` `query_prometheus_histogram` `alerting_rules_read`

The model doesn't guess numbers. It runs the same PromQL you just learned.

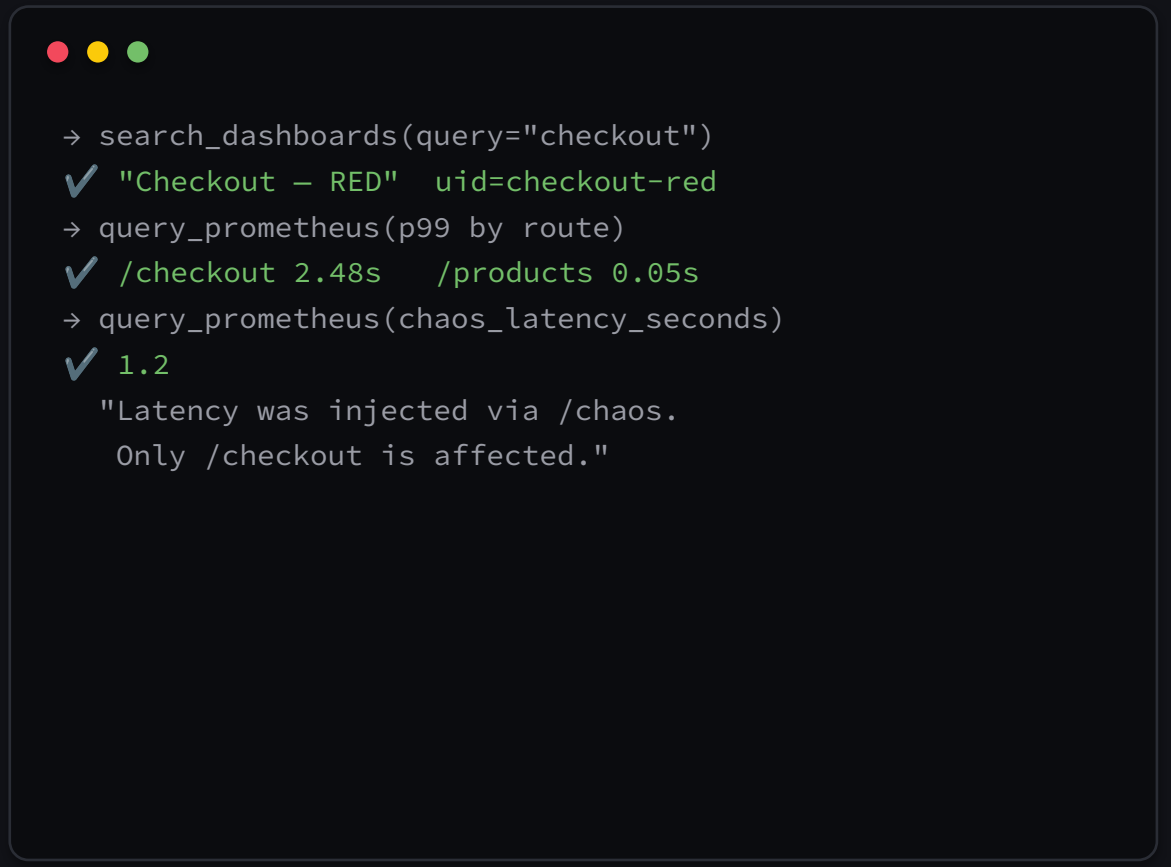
Source: MCP: What is the Model Context Protocol? ↗ · MCP spec: Tools ↗

LIVE DEMO

"Why is checkout slow?"

PROMPT

Checkout feels slow. Using Grafana, find out what changed in the last 15 minutes and why.



tool calls verified · final answer from your dry run

Source: grafana/mcp-grafana v2.0.2 README ↗

GUARDRAILS

Before you point an LLM at prod

TOKEN

Viewer only

Service-account token with the Viewer role. Nothing more.

TOOLS

Read tools only

--disable-write --enabled-tools: 19 tools, not 100+.

TRANSPORT

stdio, not open SSE

The image defaults to SSE on 0.0.0.0 with no auth.

VERIFY

Check its queries

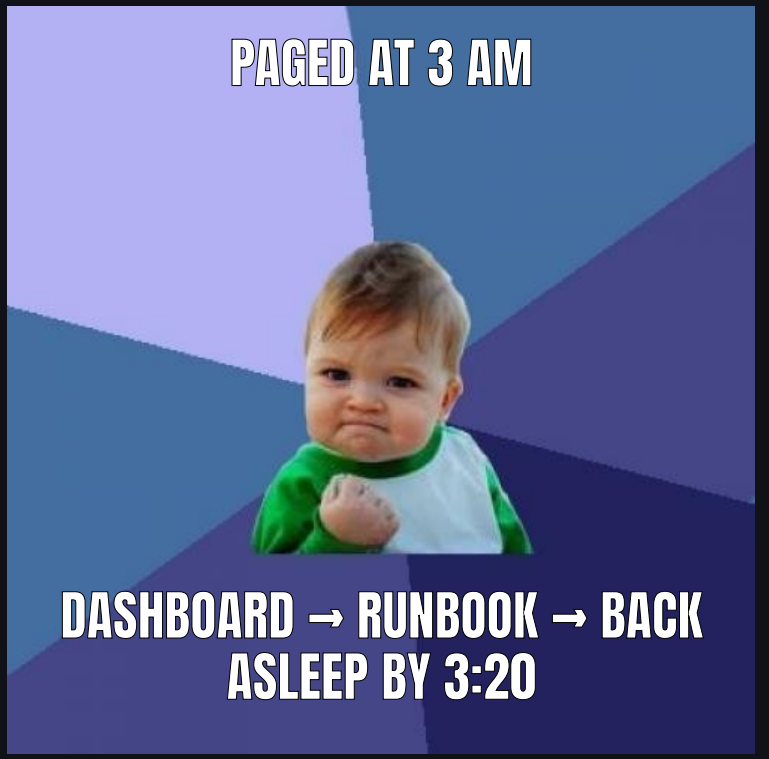
It shows the PromQL. Read it before you act on it.

Source: Grafana docs: Service accounts ↗ · grafana/mcp-grafana v2.0.2 README (flags) ↗

WHAT YOU BUILT TODAY

This morning you had logs.

- ✓ Ran an instrumented service in Docker
- ✓ Scraped it and asked questions in PromQL
- ✓ Built a RED dashboard in Grafana
- ✓ Fired — and resolved — a real alert
- ✓ Watched an LLM debug it through MCP



meme: Success Kid · imgflip

WHERE NEXT

Pick your next rabbit hole

OPEN LAB · NOW

Kubernetes

Same app on kind. Folder: kind/

LOGS

Loki

LogQL feels like PromQL for log lines.

TRACES

Tempo + OTel

Follow one request across services.

SCALE

Mimir / Thanos

Long-term storage and HA for Prometheus.

RELIABILITY

SLOs

Error budgets and burn-rate alerts.

COMMUNITY

Grafana & Friends

Monthly meetups in Mumbai. Come speak!

Source: Kubernetes: kind quick start ↗ · Loki docs: LogQL ↗

Questions?

No question is too basic. The open lab runs till 1:30.

THANK YOU, MUMBAI

Keep your dashboards green.

Cloud Native Mumbai

linktr.ee/cncfmumbai



Grafana & Friends Mumbai

linktr.ee/grafanaandfriendsmumbai



Afzal Hassan

linkedin.com/in/iemafzalhassan



REFERENCES • 1

Where every claim comes from

Docker Compose v5.5.1 release
github.com/docker/compose/releases/tag/v5.5.1

OpenTelemetry: Observability primer
opentelemetry.io/docs/concepts/observability-primer/

Grafana Labs: What is observability?
grafana.com/blog/2022/07/01/what-is-observability-best-practi…

OpenTelemetry: What is OpenTelemetry?
opentelemetry.io/docs/what-is-opentelemetry/

Prometheus docs: OTLP receiver
prometheus.io/docs/guides/opentelemetry/#enable-the-otlp-rece…

Grafana docs: Data sources (Loki, Tempo)
grafana.com/docs/grafana/latest/datasources/

Prometheus docs: Overview (pull over HTTP)
prometheus.io/docs/introduction/overview/

Grafana docs: Alerting fundamentals
grafana.com/docs/grafana/latest/alerting/fundamentals/

Docker docs: Networking in Compose
docs.docker.com/compose/how-tos/networking/

Docker docs: Compose ports (host IP binding)
docs.docker.com/reference/compose-file/services/#ports

Docker docs: Pin image versions
docs.docker.com/build/building/best-practices/#pin-base-image…

Prometheus v3.15.0 release
github.com/prometheus/prometheus/releases/tag/v3.15.0

Grafana v13.2.4 release
github.com/grafana/grafana/releases/tag/v13.2.4

Prometheus docs: Exposition formats
prometheus.io/docs/instrumenting/exposition_formats/

Prometheus docs: up metric
prometheus.io/docs/concepts/jobs_instances/#automatically-gen…

Prometheus docs: When to use the Pushgateway
prometheus.io/docs/practices/pushing/

Prometheus docs: Storage
prometheus.io/docs/prometheus/latest/storage/#local-storage

Prometheus docs: Alerting rules
prometheus.io/docs/prometheus/latest/configuration/alerting_r…

Prometheus docs: Recording rules
prometheus.io/docs/practices/rules/

Prometheus docs: Alertmanager
prometheus.io/docs/alerting/latest/alertmanager/

REFERENCES • 2

Where every claim comes from

Thanos: Quick tutorial
thanos.io/tip/thanos/quick-tutorial.md/

Grafana Mimir: HA deduplication
grafana.com/docs/mimir/latest/configure/configure-high-availa…

Prometheus docs: Configuration (defaults)
prometheus.io/docs/prometheus/latest/configuration/configurat…

Prometheus docs: kubernetes_sd_config
prometheus.io/docs/prometheus/latest/configuration/configurat…

Prometheus Operator: Introduction
prometheus-operator.dev/docs/getting-started/introduction/

Prometheus docs: Data model
prometheus.io/docs/concepts/data_model/

Prometheus docs: Metric types
prometheus.io/docs/concepts/metric_types/

Prometheus docs: Histograms and summaries
prometheus.io/docs/practices/histograms/#quantiles

Prometheus docs: Native histograms
prometheus.io/docs/specs/native_histograms/

prometheus_client (Python): Histogram
prometheus.github.io/client_python/instrumenting/histogram/

Prometheus docs: Naming, labels
prometheus.io/docs/practices/naming/#labels

Prometheus docs: Do not overuse labels
prometheus.io/docs/practices/instrumentation/#do-not-overuse-…

Prometheus docs: Aggregation operators (topk, count)
prometheus.io/docs/prometheus/latest/querying/operators/#aggr…

Prometheus docs: rate()
prometheus.io/docs/prometheus/latest/querying/functions/#rate

Prometheus docs: irate()
prometheus.io/docs/prometheus/latest/querying/functions/#irate

Prometheus docs: increase()
prometheus.io/docs/prometheus/latest/querying/functions/#incr…

Grafana docs: $__rate_interval
grafana.com/docs/grafana/latest/datasources/prometheus/templa…

Prometheus docs: Regular expressions
prometheus.io/docs/prometheus/latest/querying/basics/#regular…

Prometheus docs: histogram_quantile()
prometheus.io/docs/prometheus/latest/querying/functions/#hist…

Prometheus docs: Errors of quantile estimation
prometheus.io/docs/practices/histograms/#errors-of-quantile-e…

REFERENCES • 3

Where every claim comes from

Grafana plugin catalog: data sources
grafana.com/grafana/plugins/?type=datasource

Grafana docs: Visualizations
grafana.com/docs/grafana/latest/panels-visualizations/visuali…

Grafana docs: Provisioning
grafana.com/docs/grafana/latest/administration/provisioning/

Tom Wilkie: The RED Method
grafana.com/blog/2018/08/02/the-red-method-how-to-instrument-…

Brendan Gregg: The USE Method
www.brendangregg.com/usemethod.html

Google SRE book: Four golden signals
sre.google/sre-book/monitoring-distributed-systems/#xref_moni…

Grafana docs: Prometheus query editor (legend)
grafana.com/docs/grafana/latest/datasources/prometheus/query-…

Grafana docs: Standard options (units)
grafana.com/docs/grafana/latest/panels-visualizations/configu…

Grafana docs: Prometheus template variables
grafana.com/docs/grafana/latest/datasources/prometheus/templa…

Grafana docs: Multi-value variables
grafana.com/docs/grafana/latest/dashboards/variables/add-temp…

Grafana docs: Provisioning dashboards
grafana.com/docs/grafana/latest/administration/provisioning/#…

Grafana docs: Export a dashboard as code
grafana.com/docs/grafana/latest/dashboards/share-dashboards-p…

Grafana Foundation SDK
grafana.github.io/grafana-foundation-sdk/

Grafana Terraform provider
github.com/grafana/terraform-provider-grafana

Grafonnet
github.com/grafana/grafonnet

Prometheus docs: Alerting rules (for)
prometheus.io/docs/prometheus/latest/configuration/alerting_r…

Prometheus docs: Inspecting alerts
prometheus.io/docs/prometheus/latest/configuration/alerting_r…

Google SRE book: Symptoms vs causes
sre.google/sre-book/monitoring-distributed-systems/

Google SRE workbook: Multiwindow, multi-burn-rate alerts
sre.google/workbook/alerting-on-slos/#6-multiwindow-multi-bur…

Grafana Cloud docs: SLO burn-rate alerts
grafana.com/docs/grafana-cloud/observe-and-act/alert-and-meas…

REFERENCES • 4

Where every claim comes from

Sloth
github.com/slok/sloth

MCP: What is the Model Context Protocol?
modelcontextprotocol.io/docs/getting-started/intro

MCP spec: Tools
modelcontextprotocol.io/specification/2025-06-18/server/tools

grafana/mcp-grafana v2.0.2 README
github.com/grafana/mcp-grafana/blob/v2.0.2/README.md

Claude Code docs: MCP
code.claude.com/docs/en/mcp

Cursor docs: MCP
cursor.com/docs/context/mcp

Grafana docs: Service accounts
grafana.com/docs/grafana/latest/administration/service-accounts/

grafana/mcp-grafana v2.0.2 Dockerfile (SSE default)
github.com/grafana/mcp-grafana/blob/v2.0.2/Dockerfile

MCP spec: Transports
modelcontextprotocol.io/specification/2025-06-18/basic/transp…

Kubernetes: kind quick start
kind.sigs.k8s.io/docs/user/quick-start/

Loki docs: LogQL
grafana.com/docs/loki/latest/query/

Google SRE workbook: Alerting on SLOs
sre.google/workbook/alerting-on-slos/

Meme template: Gru's Plan
imgflip.com/memetemplate/131940431/Grus-Plan

Meme template: Left Exit 12 Off Ramp
imgflip.com/memetemplate/124822590/Left-Exit-12-Off-Ramp

Meme template: Is This A Pigeon
imgflip.com/memetemplate/100777631/Is-This-A-Pigeon

Meme template: Success Kid
imgflip.com/memetemplate/61544/Success-Kid